

LISTA DE EXERCÍCIOS DE REVISÃO — N1

Desenvolvimento de Software para Dispositivos Móveis

Engenharia de Software — UniRV

Código fonte de apoio para as questões 1 até 4

```
package com.example.livraria

class MainActivity : ComponentActivity() {
    override fun onCreate(
        savedInstanceState: Bundle?
    ) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            LivrariaTheme {
                Scaffold(
                    modifier = Modifier
                        .fillMaxSize()
                ) { innerPadding ->
                    TelaInicial(
                        modifier = Modifier
                            .padding(innerPadding)
                    )
                }
            }
        }
    }
}

data class Livro(
    val titulo: String,
    val autor: String,
    val preco: Double,
    val avaliacao: Int,
    val capa: String
)

@Composable
fun LivroCard(
    titulo: String,
    autor: String,
    preco: Double,
    avaliacao: Int,
    capa: String,
    modifier: Modifier = Modifier
) {
    Card(
        modifier = modifier
            .fillMaxWidth()
            .padding(8.dp),
        shape = RoundedCornerShape(12.dp),
        colors = CardDefaults.cardColors(
            containerColor =
                Color(0xFF5F5DC)
        ),
        elevation =
            CardDefaults.cardElevation(
                defaultElevation = 4.dp
            )
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(12.dp),
            horizontalAlignment =
                Alignment.CenterHorizontally
        ) {
            Image(
                painter =
                    painterResource(id = capa),
                contentDescription =
                    "Capa de $titulo",
                modifier = Modifier
                    .height(180.dp)
                    .fillMaxWidth(),
                contentScale =
                    ContentScale.Fit
            )
            Spacer(
                modifier =
                    Modifier.height(8.dp)
            )
            Text(
                text = titulo,
                fontSize = 14.sp,
                fontWeight = FontWeight.Bold,
                color = Color(0xFF333333),
                textAlign = TextAlign.Center
            )
            Text(
                text = autor,
                fontSize = 12.sp,
                color = Color(0xFF777777)
            )
            Text(
                text = "R$ ${ "%.2f"
                    .format(preco) }",
                fontSize = 14.sp,
                fontWeight = FontWeight.Bold,
                color = Color(0xFF2E7D32),
                modifier = Modifier
                    .padding(top = 6.dp)
            )
            Row(
                modifier = Modifier
                    .padding(top = 4.dp)
            ) {
                repeat(avaliacao) {
                    Text(
                        text = "estrela",
                        fontSize = 12.sp,
                        color =
                            Color(0xFFFFC107)
                    )
                }
            }
        }
    }
}

@Composable
fun ListaLivros() {
    val livros = listOf(
        Livro(
            titulo = "O Senhor dos Aneis",

```

```

        autor = "J.R.R. Tolkien",
        preco = 59.90,
        avaliacao = 5,
        capa = "capa_senhor"
    ),
    Livro(
        titulo = "Dom Casmurro",
        autor = "Machado de Assis",
        preco = 29.90,
        avaliacao = 4,
        capa = "capa_dom"
    ),
    Livro(
        titulo = "1984",
        autor = "George Orwell",
        preco = 39.90,
        avaliacao = 5,
        capa = "capa_1984"
    )
)

LazyColumn(
    modifier = Modifier.fillMaxSize(),
    contentPadding =
        PaddingValues(16.dp),
    verticalArrangement =
        Arrangement.spacedBy(12.dp)
) {
    items(livros) { livro ->
        LivroCard(
            titulo = livro.titulo,
            autor = livro.autor,
            preco = livro.preco,
            avaliacao = livro.avaliacao,
            capa = livro.capa
        )
    }
}
}

}

}

@Composable
fun TelaInicial(modifier: Modifier) {
    Column(modifier = modifier) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            horizontalArrangement =
                Arrangement.Center
        ) {
            Text(
                text = "Livraria ",
                fontSize = 20.sp,
                fontWeight = FontWeight.Bold
            )
            Text(
                text = "Virtual",
                fontSize = 20.sp,
                color = Color(0xFF6A1B9A)
            )
        }
        ListaLivros()
    }
}

@Preview(showBackground = true)
@Composable
fun LivrariaPreview() {
    LivrariaTheme {
        TelaInicial(
            modifier = Modifier
        )
    }
}
}

```

Questão 1 — No código de apoio, diversas funções como LivroCard, ListaLivros e TelaInicial são marcadas com a anotação @Composable. Explique qual é a finalidade dessa anotação no Jetpack Compose e por que uma função comum (sem @Composable) não poderia ser utilizada para descrever elementos de interface.

Questão 2 — Analisando a função TelaInicial no código de apoio, percebe-se o uso combinado de Column e Row. Responda:

- a) Qual é a diferença de comportamento entre Column e Row na organização dos elementos filhos?

- b) No trecho do Row que exibe o título, qual é a função do parâmetro `horizontalArrangement = Arrangement.Center`?
- c) Explique o que faz o componente `Spacer` utilizado dentro do `LivroCard` e por que ele é útil na construção de layouts.

Questão 3 — No código de apoio, observa-se o uso frequente das unidades `dp` (ex: `8.dp`, `12.dp`, `180.dp`) e `sp` (ex: `14.sp`, `12.sp`, `20.sp`). Responda:

- a) Explique a diferença entre `dp` e `sp` e indique para qual tipo de elemento cada unidade é mais adequada.
- b) Imagine que um desenvolvedor utilizou `180px` em vez de `180.dp` para a altura da imagem da capa do livro. Descreva o que aconteceria ao executar o aplicativo em dois dispositivos com densidades de tela diferentes (ex: `mdpi` e `xxxhdpi`).

Questão 4 — No código de apoio, existe a função `LivrariaPreview()` anotada com `@Preview(showBackground = true)` e a classe `MainActivity` que herda de `ComponentActivity`. Responda:

- a) Explique qual dos dois trechos é utilizado durante o **desenvolvimento** (para pré-visualização na IDE) e qual é invocado quando o aplicativo é executado no **dispositivo real**.
- b) O que aconteceria se a função `LivrariaPreview()` fosse removida do código? O aplicativo pararia de funcionar no celular? Justifique.

Questão 5 — Dispositivos móveis modernos possuem diversos sensores embutidos, como Acelerômetro, Giroscópio, Magnetômetro, GPS, sensor de proximidade e sensor de luminosidade. Responda:

- a) Explique a função de cada um dos três sensores: **Acelerômetro**, **Giroscópio** e **Magnetômetro**.
- b) Uma equipe pretende desenvolver um aplicativo de **nível digital** (bolha) para smartphones, que indica se uma superfície está perfeitamente nivelada. Qual sensor seria o principal para essa funcionalidade? Justifique.
- c) Em um aplicativo de **bússola digital**, qual sensor seria o mais adequado? Justifique.

Questão 6 — Analise o seguinte trecho de código Kotlin:

```
val multiplicar = { x: Int, y: Int -> x * y }  
val resultado = multiplicar(4, 7)  
println(resultado)
```

- a) Explique o que é uma **função lambda** (função anônima) em Kotlin e por que dizemos que funções são “cidadãs de primeira classe” nessa linguagem.
- b) No código acima, o que a variável `multiplicar` armazena e qual será o valor impresso pelo `println`?
- c) Reescreva o trecho acima criando uma lambda que receba uma `String` e retorne essa string em letras maiúsculas.

Questão 7 — Considere o trecho de código Kotlin abaixo:

```
val frutas = mutableListOf("Manga", "Uva", "Manga", "Abacaxi")  
val conjunto = mutableSetOf("Manga", "Uva", "Manga", "Abacaxi")  
val mapa = mutableMapOf("A" to 1, "B" to 2, "A" to 3)
```

- a) Descreva as principais diferenças entre `List`, `Set` e `Map` em Kotlin.
- b) Qual será o valor de `frutas.size`, `conjunto.size` e `mapa.size`? Justifique cada resposta.
- c) Explique a diferença entre as variantes `mutableListOf` e `listOf`. A declaração com `val` impede a adição de novos elementos a uma coleção mutável?

Questão 8 — Sobre as funções de alta ordem `filter` e `map` em Kotlin, responda:

- a) Explique o que cada uma dessas funções faz e se elas alteram a coleção original ou retornam uma nova coleção.
- b) Qual é o resultado das expressões abaixo? Justifique.

```
val numeros = listOf(1, 2, 3, 4, 5, 6)
val pares = numeros.filter { it % 2 == 0 }
val dobrados = numeros.map { it * 2 }
println(pares)
println(dobrados)
```

- c) O que é o identificador implícito `it` utilizado dentro das lambdas acima?
- d) Escreva uma expressão que, a partir de `numeros`, retorne uma lista contendo apenas os valores maiores que 3, multiplicados por 10 (encadeando `filter` e `map`).

Questão 9 — No código de apoio, a classe `Livro` é declarada como uma `data class`. Explique o que diferencia uma `data class` de uma classe comum em Kotlin. Quais métodos são gerados automaticamente pelo compilador ao utilizar essa palavra-chave?

Questão 10 — No Jetpack Compose, o conceito de `Modifier` é amplamente utilizado, como pode ser visto no código de apoio (ex: `Modifier.fillMaxWidth()`, `Modifier.padding(8.dp)`, `Modifier.height(180.dp)`). Responda:

- a) O que é um `Modifier` no Jetpack Compose e qual a sua finalidade?
- b) Por que diversos `Modifiers` podem ser encadeados (ex: `Modifier.fillMaxWidth().padding(8.dp).height(180.dp)`). Que padrão de projeto isso lembra?

Questão 11 — No código de apoio, a função `ListaLivros()` utiliza um `LazyColumn` para exibir os cards de livros. Explique a diferença entre `LazyColumn` e `Column` e em quais cenários o uso de `LazyColumn` é preferível.

Questão 12 — Considerando a lista de livros abaixo em Kotlin:

```
data class Livro(  
    val titulo: String,  
    val autor: String,  
    val preco: Double,  
    val avaliacao: Int  
)  
  
val livros = listOf(  
    Livro("Kotlin em Acao", "Dmitry", 89.90, 5),  
    Livro("Clean Code", "Robert", 69.90, 4),  
    Livro("O Hobbit", "Tolkien", 34.90, 5),  
    Livro("Design Patterns", "GoF", 120.00, 3)  
)
```

Escreva trechos de código Kotlin que realizem as seguintes operações utilizando funções de alta ordem:

- a) Filtrar somente os livros com preço menor que R\$ 70,00.
- b) Obter uma lista contendo apenas os títulos dos livros com avaliação igual a 5.
- c) Calcular o preço total de todos os livros da lista (dica: use `sumOf`).

Boa revisão!